

G1_29 Payload Gravity Compensation 与在线质量估计结构说明

xr_teleoperate

2026-04-16

问题定位

这次改动的重点不是“给手指单独做重力补偿”，而是：

- 把手里抓取物体的重量建模为末端执行器上的 payload 外载
- 把 payload 引起的 ee 外力 / 外力矩映射成整条手臂的关节前馈力矩
- 在此基础上补齐观测链，为未知质量估计预留结构

当前控制链路：

$\text{solve_ik}(\cdot) \rightarrow \text{pin.rnea}(\cdot) \rightarrow \text{sol_tauff} \rightarrow \text{arm controller} \rightarrow \text{motor_cmd[id]}.tau$

总体改动分三阶段

阶段一：已知质量的静态 / 准静态 payload gravity compensation

- 只支持 G1_29
- 在 IK 前馈链中新增 `tau_payload`
- 输出改为 `sol_tauff = tau_nominal + tau_payload`

阶段二：补齐观测链

- low state 中新增 `tau_est`
- 主循环组装 `arm_obs={q,dq,tau_est}`

阶段三：新增 PayloadEstimatorG1_29

- 在线输出 `mass_hat`
- 当前只打印 / 记录，尚未闭环写回补偿

为什么不是“给手指单独做重力补偿”

payload 问题的主物理本质是：

- 末端抓着一个有质量的物体
- 物体重力首先表现为 ee 上的外力 / 外力矩
- ee 外载再沿 wrist / elbow / shoulder 传递为整条手臂的关节力矩需求

因此补偿应落在 **arm torque feedforward**，而不是先改 hand controller。

第一版也没有直接做 whole-body，原因是：

- 当前最稳定的入口是 dual-arm feedforward
- whole-body 会引入 torso / waist / contact / base coupling
- 会显著扩大建模和实机回归风险

阶段一：补偿挂点与参数入口

主脚本新增 G1_29 专用配置：

- `payload_enable`, `payload_side`
- `payload_mass`, `payload_com`
- `payload_scale`
- `payload_debug`, `payload_log_every`
- `arm_tau_limit`

在 IK 层：

$$\tau_{nominal} = rnea(q, \dot{q}, 0)$$

$$\tau_{payload} = J^{\top} w_{payload}$$

$$sol_tauff = \tau_{nominal} + \tau_{payload}$$

payload gravity compensation 原理

payload 的质心 `payload_com` 定义在 active side 的 `L_ee` / `R_ee` frame 下。
在世界系中：

$$f = \begin{bmatrix} 0 \\ 0 \\ -mg \end{bmatrix}$$
$$m = r \times f$$

于是 ee 上的 wrench 为：

$$w_{payload} = [m_x \quad m_y \quad m_z \quad f_x \quad f_y \quad f_z]^\top$$

实现采用 `LOCAL_WORLD_ALIGNED` Jacobian，并使用 Pinocchio wrench 顺序

$$[m_x, m_y, m_z, f_x, f_y, f_z]$$

为什么 $\tau_{payload} = J^T w$

根据虚功关系：

$$\delta W = w^T \delta x = \tau^T \delta q, \quad \delta x = J \delta q$$

所以有：

$$\tau = J^T w$$

物理含义：

- payload 的重量不是停留在笛卡尔空间里的一个“末端力”
- 它最终会转化为各个关节需要承担的附加支撑力矩
- 所以补偿目标必须落到 joint torque space

当前 G1_29 reduced model 为 14 维:

- 左臂: 0:7
- 右臂: 7:14

第一版明确只对 active side 生效:

- 抓左手物体, 只保留左臂补偿
- 抓右手物体, 只保留右臂补偿

安全处理上:

- IK 层只负责输出理论 `sol_tau`
- controller 层在真正写 `motor_cmd[id].tau` 前做 clip

这样把“算法输出”和“安全护栏”明确分开。

阶段二：为什么要补 τ_{est}

只有 q/dq 时，我们能知道：

- 当前姿态与速度
- 名义模型 torque 应该是多少

但我们不知道：

- 电机实际承受了多少 torque
- 模型预测与真实系统之间差了多少

所以第二阶段补齐：

$$\tau_{residual} = \tau_{est} - \tau_{nominal}$$

未知质量估计依赖的核心信息，正是这个“模型未解释掉的力矩残差”。

阶段三：PayloadEstimatorG1_29 的定位

当前估计器的设计范围：

- 只支持 G1_29
- 只估计单个标量 `mass_hat`
- 固定 `payload_com`
- 只做 active side 估计
- 暂时不闭环写回补偿

这意味着它当前更像一个“验证型估计器”：

- 验证残差是否真的反映 `payload` 质量
- 验证现有观测条件下质量估计是否稳定
- 为下一步闭环补偿提供基础

为什么第一版只估 `mass_hat`

若同时估计质量和质心：

- 参数维度更高
- 质量与几何误差相互耦合
- 可辨识性和稳定性都会明显变差

因此第一版固定 `payload_com`，只估一个标量质量：

- 先把几何假设固定
- 先验证“质量这一维”是否能稳定识别
- 后续再考虑扩展到 `com_hat`

质量估计公式

在估计器内部，使用静态 / 准静态近似：

$$\tau_{nominal} = r_{nea}(q, 0, 0)$$

构造单位质量模板：

$$\tau_{unit}(q) = J^T w_{unit}$$

其中：

$$\tau_{residual} = \tau_{est} - \tau_{nominal}$$

$$\tau_{residual} \approx m \cdot \tau_{unit}$$

对应的一维最小二乘闭式解为：

$$m_{raw} = \frac{\tau_{unit}^T \tau_{residual}}{\tau_{unit}^T \tau_{unit}}$$

本质上是在 joint torque space 中，把残差投影到 “payload 重力方向模板” 上。

为什么需要门控、限幅与平滑

实机中的 τ_{est} 会混入噪声、摩擦和动态项，因此需要稳健性处理：

- 可观测性门控：防止 $\tau_{unit}^\top \tau_{unit}$ 太小导致病态放大
- dq 低速门控：保证静态 / 准静态近似仍然成立
- 合法性检查：过滤 NaN / inf / 缺失数据
- EMA 平滑：抑制单帧抖动
- 限幅与 hold / fallback：保证输出在合理质量区间内，并在失效时保持稳定

当前方案的边界和局限

- 只支持 G1_29
- 只支持单手 active side payload
- 只适合静态 / 准静态 payload 场景
- 还没有估计 com_hat
- 还没有引入 whole-body coupling
- 估计器还没有闭环写回补偿链

因而当前阶段的准确定位是：

先验证补偿链、观测链、估计链是否物理一致且工程可用，再推进到真正的在线自适应 payload compensation。

仓库内直接实现参考

- teleop/teleop_hand_and_arm.py
- teleop/robot_control/robot_arm_ik.py
- teleop/robot_control/robot_arm.py
- teleop/robot_control/payload_estimator.py

外部参考

- Pinocchio 官方仓库与文档: RNEA、frame Jacobian、ReferenceFrame 语义
- Carpentier et al., *The Pinocchio C++ library*, SII 2019
- Roy Featherstone, *Rigid Body Dynamics Algorithms*, Springer, 2008
- Lynch & Park, *Modern Robotics*, Chapter 5: Velocity Kinematics and Statics

这些资料支撑的是标准动力学 / 静力学原理;

而当前三阶段 payload 结构是面向本仓库约束做出的工程集成设计。

总结

这次工作的重点，不是单独“做了一个 estimator”，而是把 G1_29 的 payload 能力从
已知质量的静态补偿

推进到

具备在线质量估计结构的工程化补偿框架

三个关键里程碑：

- 已知质量补偿链已接入
- τ_{est} 观测链已补齐
- $mass_{hat}$ 在线估计模块已就位